PL/pgSQL Programming

FitZone Gym Management System — Follow-up Assignment

1. Continuation of the FitZone Project

This assignment builds directly on the fitzone_db database you already created and populated in the previous assignment. You will keep working on the exact same 6 tables and the exact same data — members, trainers, categories, classes, sessions, and bookings — no schema redesign is needed.

You're free to add any extra supporting table(s) if your design calls for them (e.g. audit_log in Part B, or anything else you think improves your solution) — just keep the original 6 core tables intact, and add a short comment explaining why you introduced each new table.

All PL/pgSQL objects (blocks, functions, procedures, triggers) must be written against this schema, and must actually run successfully on your populated data — a function that only works on paper (or throws errors on your real rows) will not receive credit.

2. Requirements

The requirements are split into two parts, following the same order as the two lectures. Complete each phase in order and document your work in one organized SQL file with comments explaining each section.

Part A — PL/pgSQL Basics

Phase 1: Blocks & Variables

1. Write an anonymous DO block with a full DECLARE / BEGIN / EXCEPTION / END structure that declares a variable using %TYPE against members.email, retrieves the email of the member with the lowest member id using SELECT INTO, and prints it with RAISE NOTICE.
2. In the same block, declare a second variable (a plain type, e.g. INTEGER) to hold that member's total number of bookings, calculate it with a query, and print both values together.
3. Write a short note (2–3 sentences) on the difference between running this logic as a plain SQL SELECT versus as a PL/pgSQL block — what procedural capability did you gain?

Phase 2: Control Flow & Loops

4. Write a DO block that checks a specific member's status column with IF / ELSIF / ELSE and RAISE NOTICE a different message for 'active', 'inactive', and any other value.
5. Write a FOR loop that iterates over all classes belonging to one specific category and prints each class name.
6. Write a WHILE loop that counts how many sessions a specific trainer has led, incrementing a counter variable until all their sessions have been checked (do not just use COUNT() directly — the point is to practice the loop).

Phase 3: Functions

7. get_member_full_name (RETURNS TEXT) — returns the member's full name (first + last).
8. get_average_rating_by_trainer (RETURNS NUMERIC)— returns the average booking rating across all sessions led by that trainer.
9. calculate_loyalty_points(p_member_id INT) RETURNS INT — calculates loyalty points as (number of bookings with a rating 5 .
   This function is required to calculate the loyalty points based on the number of bookings made by a specific member with a rating of 5 in Bookings table.

For each function, show the CREATE FUNCTION statement and a sample SELECT calling it.

Phase 4: Procedures

10. add_new_booking(p_member_id INT, p_session_id INT) — inserts a new row into bookings for that member/session (rating and comment left NULL, to be filled in later).
11. update_member_status(p_member_id INT, p_new_status VARCHAR) — updates a member's status column.

Phase 5: Error Handling & RECORD Variables

12. Write a DO block that uses a RECORD variable inside a FOR loop to iterate through every session of one class, printing the session date, room, and trainer id from the record.
13. Extend add_new_booking (or write a new version) so that the INSERT is wrapped in a BEGIN / EXCEPTION block. Deliberately trigger an error on purpose (e.g. an invalid session_id) and show that your EXCEPTION block catches it and prints a friendly message instead of crashing.

Phase 6: Practical Applications of PL/pgSQL

14. User Lookup: get_member_by_email- (or use OUT parameters) — returns the full member row for a given email, or a clear message if not found.
15. Data Validation: is_valid_rating -RETURNS BOOLEAN — returns TRUE only if the rating is between 1 and 5.
16. Batch Operation: apply_loyalty_bonus() — loops through every member with more than 3 bookings and increases their loyalty level by 1, without allowing the value to exceed 5
17. Business Logic: a function that, given a session_id, returns how many available seats remain (assume a fixed room capacity of your choice, e.g. 15, defined as a constant inside the function).

Part B — Advanced PL/pgSQL

Phase 7: Named Exception Handling

18. Write a DO block or procedure that attempts to insert a new trainer with an email that already exists, and catches the error specifically using the unique_violation named exception, printing a friendly message instead of letting PostgreSQL raise the raw error.
19. Write a second block that attempts to insert a booking with a session_id that does not exist, and catches it specifically using the foreign_key_violation named exception.

Phase 8: Additional PL/pgSQL Practice

20. Write a DO block that declares an INTEGER variable, assigns it the total number of members in the members table using SELECT INTO, and prints the result using RAISE NOTICE

Phase 9: Triggers — Validation & Business Rules

21. Create a two trigger:
    A. Validation Trigger: choose ONE (rating 1-5 OR prevent duplicate booking).
    B. Business Rule Trigger: prevent updating rating twice.

Phase 10: Audit Logging

22- Create the audit_log table and implement a trigger function that logs INSERT, UPDATE, and DELETE operations on the bookings table.

Phase 11: Business Rules in the Database

23- Create an AFTER UPDATE trigger on bookings that automatically increases the corresponding member's loyalty level by 1 whenever a NULL rating is filled in for the first time (i.e. the rating changes from NULL to a real value). The loyalty level must never exceed 5.

3. Notes & Best Practices (Must Follow)

- Function and procedure names: lowercase, underscore-separated, verb-first where it makes sense (get_..., calculate_..., add_..., update_...).
- Use RAISE NOTICE (not RAISE EXCEPTION) for informational messages; reserve RAISE EXCEPTION / named exceptions for actual error conditions.
- Test every function and procedure against your real FitZone data before submitting — include the sample call .
- Keep the original 6 core tables as the foundation — feel free to introduce any additional supporting table(s) your design needs, as long as they're explained with a comment.
- Comment (--) each phase clearly in the SQL file so it stays easy to grade, exactly like the previous assignment.

4. Deliverables

- One SQL file named fitzone_plpgsql_assignment.sql containing all DO blocks, functions, procedures, and triggers in order, with clear comments and sample calls/output.
- Submit as a ZIP folder named BY YOUR Email after you, same as before.